

Analysis Pipeline

Rebecka Jörnsten, Mathematical Sciences

MSA220/MVE441 Statistical Learning for Big Data

27nd March 2026



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF GOTHENBURG

Importance of data splitting

Unbiased estimation of expected misclassification error rate

- ▶ You can't train/tune a model and use the same data to obtain unbiased estimates of the misclassification rate you expect to observe on future data sets
- ▶ Solution: hold-out data used only for testing

What do we need to think about?

Unbiased estimation of expected misclassification error rate

What do we need to think about?

- ▶ How much data can we "afford" to hold out for testing?
- ▶ "Too much" handicaps training
- ▶ "Too little" leads to poor estimates of the misclassification error rate

Unbiased estimation of expected misclassification error rate

What do we need to think about?

- ▶ Stratified sampling
- ▶ If there is class imbalance in your data set, use stratified sampling to ensure that class proportions are similar in both training and testing data sets.

Something to think about: could there be sampling bias in your data set as a whole?

Imbalanced data

Imbalanced data

What if some classes are less frequent than others?

- ▶ Accuracy can be very misleading as a performance metric if your data is imbalanced
- ▶ If there is a dominant majority class, you can get high accuracy by simply predicting that class for all observations!

What should we do?

Imbalanced data

What if some classes are less frequent than others? What should we do?
Use different performance metrics for validation of your methods.

- ▶ Weighted accuracy: weighted average of class specific error rates, or average class specific error rates. Weights $\sim n/(Cn_c)$
- ▶ AUC - area under ROC - trading off early detection vs false positives
- ▶ Sensitivity
- ▶ Kappa: normalizing accuracy by the accuracy you would expect to get just by chance for the imbalance in your data set
- ▶ F1: harmonic mean of precision and recall, and extensions to multiple classes (avg of class-wise precision and recall, or class-wise F1 scores)
- ▶ Matthew's correlation coefficient, MCC: correlation between binary predicted labels and true labels. Can be generalized to multiple classes as well. Takes all entries of a confusion matrix into account.

Imbalanced data

- ▶ Using different performance metrics can make a big difference in comparing methods
- ▶ Can also impact the tuning of methods substantially!

Note! You can also use a different metric to TRAIN your methods - meaning a different loss function than the standard ones. Check options in your machine learning pipeline!

A very simple option is to use a weighted loss. For example, make the minority classes more important in the training by weighing their contribution by $n/(Cn_c)$ or some other upweighting scheme.

Imbalanced data

Changing the data set.

If the data set is highly imbalanced you can also alter the *training data* to address this challenge.

- ▶ Downsample the majority classes
- ▶ Upsample the minority classes
- ▶ Augment the minority classes by generating new observations as linear combinations of a sample and a nearest neighbor sample in the class.
- ▶ Or a combination of these: SMOTE (Synthetic Minority Over-sampling Technique).

This is for *training* only - you should not rebalance the test data. You can use balanced metrics to validate though.

Data leakage

- ▶ All choices should be done on the training data, none based on the test data
- ▶ This includes data transformations, such as box-cox, PCAs etc.
- ▶ The transformations you "learn" (e.g. PC-projection, standardization etc) should be applied as trained on the test set and not re-evaluated.

Data leakage

All choices should be done on the training data, none based on the test data

- ▶ If you use a simple hold-out approach, this is fairly easy.
- ▶ If your full data set is used for exploration of the data (e.g. choosing to take logs of features etc) then there is some leakage into the CV for selecting tuning parameters.
- ▶ Choosing the number of PCs and applying trained PC projections to testing data can be implemented without data leakage in a CV-procedure since you can use the CV-batches to perform PCA etc.
- ▶ Anything that is tunable by performance metrics should be placed inside the CV-loop.
- ▶ This includes filtering - threshold on test-statistics to remove uninformative features, etc.

What if you want to repeat the train-test split, have an outer loop to assess stability/reproducibility of the tuning and testing performance?

- ▶ Use an outer 3rd loop for unbiased test error performance
- ▶ Recognize that the single hold-out procedure used for data exploration can lead to some optimism in the test error performance when you repeat the splits.

Reporting

Comparing methods

- ▶ Make sure you know how tuning is implemented for the methods you choose to compare
- ▶ What are default tuning parameter settings?
- ▶ What is the default grid-search setting for tuning parameters?
- ▶ If you don't use the tuning properly you can get very misleading results in your comparisons!
- ▶ Case in point
 - ▶ LDA when $n < p$: packages may revert to a regularized version without warning you. LogisticRegression in python has a default where coefficients are penalized with L2 norm.
 - ▶ Multinomial regression: packages may use elastic net as default
 - ▶ NN: package may require you to manually set the grid search parameters
 - ▶ RF: packages may have default setting that hamper training if there is weak signal in the first splits
 - ▶ etc etc. READ THE MANUALS/HELP FILES!!!!

Comparing methods

- ▶ Try to make direct comparisons whenever possible
- ▶ That is - if you use different train-test splits for different methods, you may not get a fair assessment
- ▶ Stability of performance is an important property to assess. If a method has the potential to perform really well, but also the potential to fail spectacularly, is this a good method to use on future data?
- ▶ Don't summarize by an overall accuracy only - make sure you look at the confusion matrices. Are some methods unable to detect some classes? Some methods better for other parts of the data. Perhaps this can help you select an even better method? (Data leakage warning here).